

Mission 3 of 4 — System Expansion

Suggested time: ~15 minutes | Deliverable: Architecture Version 2

NimbusCart's pilot launch went well. With a major regional festive sale coming up, the client wants the platform ready for a large, short-term surge in traffic — and a better shopping experience to go with it.

New / Changed Client Requirements

- Buyers should see personalized product recommendations based on their browsing and purchase history.
- The platform must stay responsive under a sudden spike in read traffic during the sale window.
- Product images and static assets should load quickly for buyers across different regions.
- The operations team wants basic monitoring and alerting in place before the sale, to catch issues early.

Constraints Update

- The 9,500 budget unit monthly cap still applies to the combined architecture — Mission 1 selections plus anything you add now.
- You may revisit and change an earlier (Mission 1 or 2) choice if a new requirement genuinely calls for it — but be ready to justify the change.
- Not every new requirement needs a brand-new component; some may be satisfied by adjusting how an existing one is used.

Task: Extend to Architecture Version 2

- Identify which new requirements need a new component from the Technology Inventory, and which can be met by adjusting your existing architecture.
- Add the components you choose to your Mission 2 diagram to produce Architecture Version 2.
- Keep the original request flow intact — you're extending the system, not starting over.
- Note briefly, next to each addition, which new requirement it addresses.

Deliverable

A completed Architecture Version 2 diagram, extending your Mission 2 design with the components needed to satisfy the new requirements, and a brief note on why each was added.

Evaluation Focus: Problem Solving and Decision Making • Software Architecture Design

■ Hints — Mission 3 – System Expansion

- Match each new requirement to the smallest architectural change that satisfies it — not every new requirement needs a brand-new tier.
- It's fine to revisit a Mission 1 or 2 choice if a new requirement genuinely demands it — just be ready to explain why.
- Keep a running total of cost as you add components; expansion is the stage where teams most often go over the budget cap.

■ Hints — General

- There is no single correct architecture. The jury is evaluating your reasoning and clarity, not whether you matched a hidden answer key.
- Re-read the client requirements and business constraints before opening the Technology Inventory — it's easy to pick a popular-sounding option that doesn't actually fit the brief.
- The 60 minutes are split across four missions. Pace yourself; don't spend so long perfecting Mission 1 that later missions get rushed.
- Every deliverable asks for a short written justification somewhere — keep a running note of *why* you chose each component, in plain language.
- Cost and complexity are trade-offs, not penalties. A more expensive or more complex option can be the right call if the requirements justify it — say so.

■ Recommendations — hosted API vs. custom-trained model

Supplementary context, not part of the official brief — useful for explaining your reasoning.

- **AI/ML Recommendation API** — reach for this when the team has no dedicated ML engineers, needs something working soon, and "good enough" personalization is fine. Zero training/hosting overhead, cost scales with usage.
- **Custom-trained model (e.g. PyTorch)** — reach for this only when the client needs recommendation quality or behavior a generic API can't provide, and the team has ML expertise, labeled data, and capacity to host and retrain a model (often on GPU instances). For a small team on a tight timeline and budget, this is usually the higher-risk, higher-cost path.

■ Glossary — terms used on this page

Term	What it means
AI/ML Recommendation API	A ready-made, hosted service that returns personalized suggestions (e.g. products) without the team having to train or host a model.
Custom-Trained ML Model (e.g. PyTorch)	A machine-learning model the team builds, trains, and hosts itself using a framework such as PyTorch or TensorFlow — full control over accuracy and behavior, but requires ML expertise, data, GPU infrastructure, and ongoing retraining.
GPU Instance	A server equipped with graphics processing hardware, used to train or run demanding machine-learning models faster.

In-Memory Cache (Redis / Memcached)	A fast, temporary data store kept in memory, used to avoid repeating expensive database reads for the same data.
Object Storage	Storage designed for large amounts of unstructured files (images, videos, backups), usually paired with a CDN for fast delivery.
CDN (Content Delivery Network)	A network of servers spread across locations that cache and serve content closer to the user, reducing load time.
APM (Application Performance Monitoring)	Tooling that tracks how an application is performing in production — response times, errors, and traces through the system.
Read Replica	A copy of a database used to handle read queries, so the primary database is freed up mainly for writes.
Kafka-class Event Stream	A high-throughput system for publishing and consuming continuous streams of events, used when many services need to react to the same events in real time.
Budget Unit / Cost Cap	The abstracted monthly cost figure assigned to each technology option in the simulation, used to check choices against the client's spending limit.